Greedy Method

- It is the simplest and straight forward algorithm design strategy.
- Greedy method is used to solve optimization problems.
- Optimization problems have **objective functions** and a set of **constraints.** The solution to this problem is to create a subset of solutions that satisfies constraints associated with the problem.
- Any subset solution that satisfies these constraints is called a **feasible solution.**
- Any feasible solution that maximizes or minimizes the given objective function is called the **optimal solution**.
- Every feasible solution need not be an optimal solution. But every optimal solution is a feasible solution.

Greedy Method

- The greedy method works in stages. At each stage, a decision or choice is made regarding whether or not a particular input to be considered for an optimal solution.
- These decisions are locally optimal. Finally the global optimal solution is obtained by combining locally optimal solutions.

Control abstraction for Greedy Method

```
Algorithm Greedy (a, n)
{
        solution:= Ø;// initialize the solution set to empty
        for i :=1 to n do
        {
           x ← Select (a);
           if Feasible (solution, x) then
                   solution:=Union (solution, x);
        }
  return (solution);
}
```

Select (a) is an algorithm which picks up an element from the array.

Feasible (solution, x) is a procedure that checks whether the element x when included in the set solution, will lead to a solution or not.

Union (solution, x) is a procedure which adds the element x to the solution set solution

Greedy Method

- **Problems that use the greedy approach to find an optimal solution are:**

1. Coin change problem
2. Job Scheduling Problem(Job sequencing)
3. Knapsack Problem
4. Minimum cost spanning trees
5. Single source shortest path problem.
6. Optimal Merge Patterns

Coin change problem

- With given coins of denominations find out a way to give a customer an amount with fewest number of coins.
- For example, if I ask you to return me change for 2000

and denominations of coins :{10,50,100,200,500}

- Amount : 2000
- Solutions :
- 1.4X 500 (4 coins)
- This solution is the optimal one as it gives us change only with 4 coins

Knapsack problem

- We are given n items and a knapsack or bag of capacity m.We need to put these items in a knapsack to get the **maximum profit** value in the knapsack.
- Here each item i has a weight w_i and profit p_i and the knapsack has a capacity m.
- If a fraction x_i, $0 \leq x_i \leq 1$ of item i is placed into the knapsack then a profit of $p_i * x_i$ is earned.
- The objective is to fill the knapsack that maximizes the total profit earned.

Knapsack problem

- Formally the problem can be stated as follows:

$$\text{maximize} \sum_{1 \le i \le n} p_i x_i \qquad \text{—— (1)}$$

$$\text{subject to} \sum_{1 \le i \le n} w_i x_i \le m \qquad \text{—— (2)}$$

$$\text{and } 0 \le x_i \le 1, \quad 1 \le i \le n \qquad \text{—— (3)}$$

- The profits and weights are positive numbers.
- A feasible solution is any set (xi,..., xn) satisfying (2)and (3)above.
- An optimal solution is a feasible solution for which (1) is maximized.

Types of Knapsack problems

- There are two types of knapsack problems:
- 1. **Fractional Knapsack problems**
- -In these problems we can break items for maximizing the total value of knapsack.
- 2. **0/1 Knapsack Problems**
- -In these problem we are not allowed to break items. We either take the whole item or don’t take it.

Knapsack problem

- Methods for selecting items in knapsack problem:
- 1. **Greedy selection based on profit**
- 2. **Greedy selection based on weight**
- 3**.Greedy selection based on profit/weight ratio**

Knapsack problem

- Consider the following instance of the knapsack problem:
- n = 3,m= 20 (p1,p2,p3) = (25,24,15), and (w1,w2,w3)= (18,15,10).
- Optimal solution is =(0,1,1/2)

  maximum profit=31.5

Four feasible solutions are:

	(x_1, x_2, x_3)	$\sum w_i x_i$	$\sum p_i x_i$
1.	$(1/2, 1/3, 1/4)$	16.5	24.25
2.	$(1, 2/15, 0)$	20	28.2
3.	$(0, 2/3, 1)$	20	31
4.	$(0, 1, 1/2)$	20	31.5

Knapsack problem

- Consider the following instance of the knapsack problem:
- n = 3,m= 15 (p1,p2,p3) = (24,18,20), and (w1,w2,w3)= (8,9,5).
- Optimal solution is =(1,2/9,1)

  maximum profit=48

Knapsack problem

Item	A	B	C	D
Profit	280	100	120	120
Weight	40	10	20	24

m=60

- profit is **100 + 280 + 120 * (10/20) = 380 + 60 = 440**

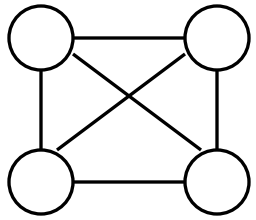
Greedy Knapsack Algorithm

```
Algorithm GreedyKnapsack (m, n)
// P[1 : n] and w[1 : n] contain the profits and weights respectively of
// Objects ordered so that p[i] / w[i] > p[i + 1] / w[i + 1].
// m is the knapsack size and x[1: n] is the solution vector.
{
        for i := 1 to n do x[i]   := 0.0                    // initialize x
        U := m;
        for i := 1 to n do
        {
                if  (w(i) > U) then break;
                x [i] := 1.0; U := U – w[i];
        }
        if (i ≤ n) then x[i] := U / w[i];
}
```

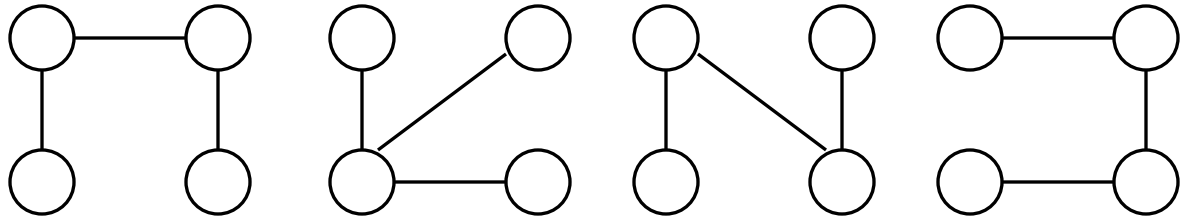
Time complexity is O(nlogn)

Minimum cost spanning trees

- Suppose you have a connected undirected graph then a spanning tree of the graph is a connected sub graph in which there are no cycles.
- If a graph has n nodes, the no. of possible spanning trees is n^{n-2}.

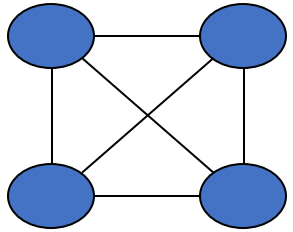


A connected, undirected graph

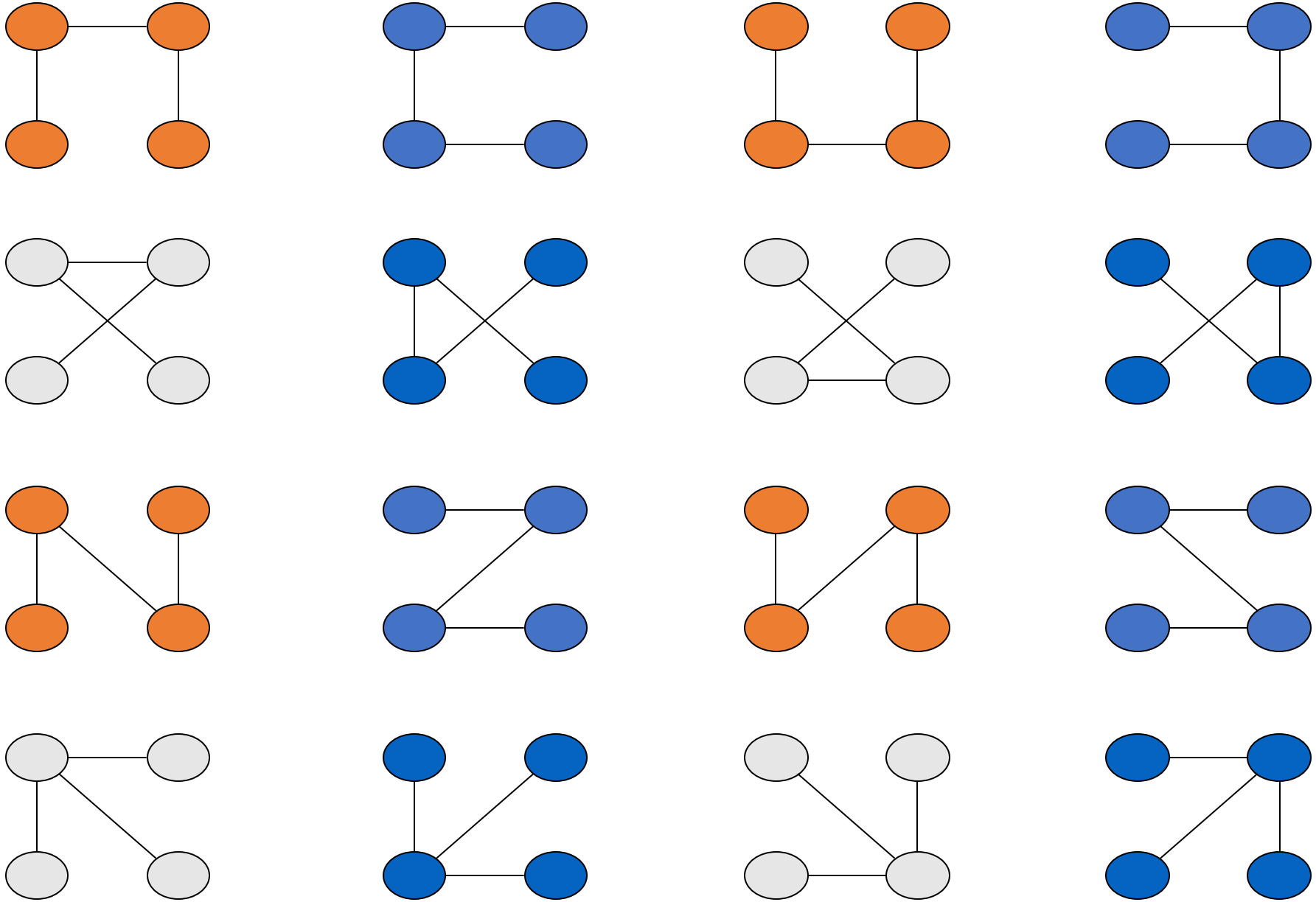


Four of the spanning trees of the graph

Complete Graph

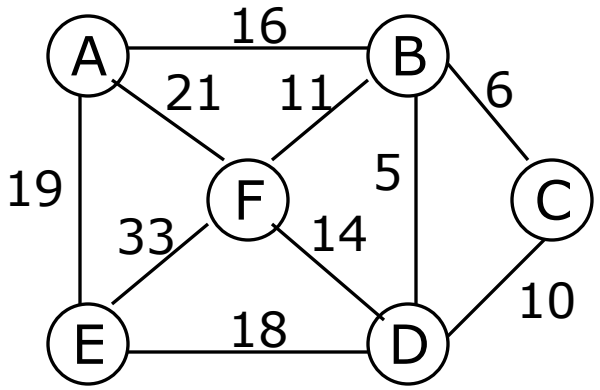


All 16 of its Spanning Trees

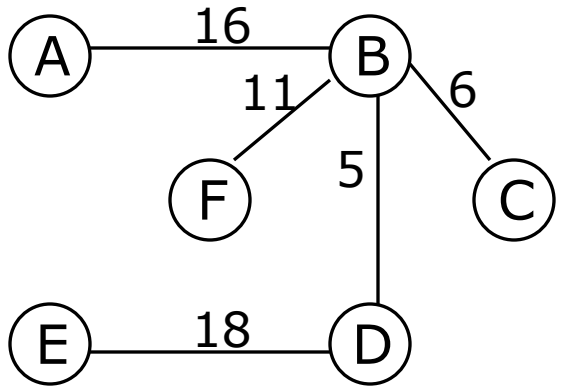


Minimum cost spanning trees

- Suppose you have a connected undirected graph with a weight (or cost) associated with each edge
- The cost of a spanning tree would be the sum of the costs of its edges.
- A minimum-cost spanning tree is a spanning tree that has the lowest cost.



A connected, undirected graph



A minimum-cost spanning tree

Minimum cost spanning trees

- There are three basic algorithms for finding minimum-cost spanning trees, and all are greedy algorithms that produce optimal solutions.

1. **Prim's algorithm**
2. **Kruskal's algorithm**
3. **An optimal randomized Algorithm**

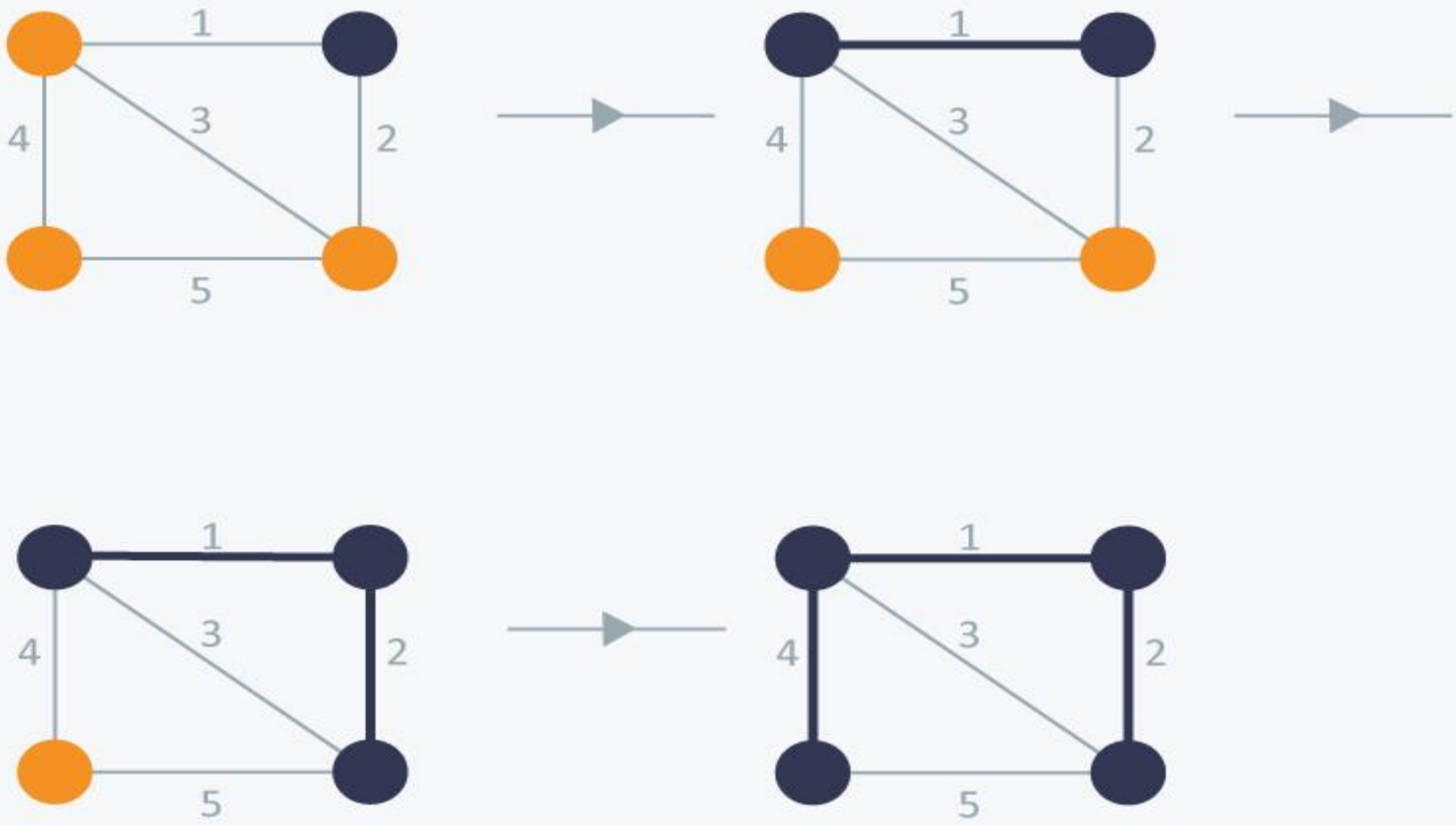
Prim’s algorithm

Steps for finding MST using Prim’s algorithm:

1. Choose a vertex V as starting vertex of the spanning tree.
2. Select an edge with minimum weight that connects V.
3. If this edge does not form a cycle ,then add it to the tree else discard it.
4. Repeat steps 2-3 until all vertices are added to the tree.

- Here, we consider the spanning tree to consists of both nodes and edges.

Prim's Algorithm

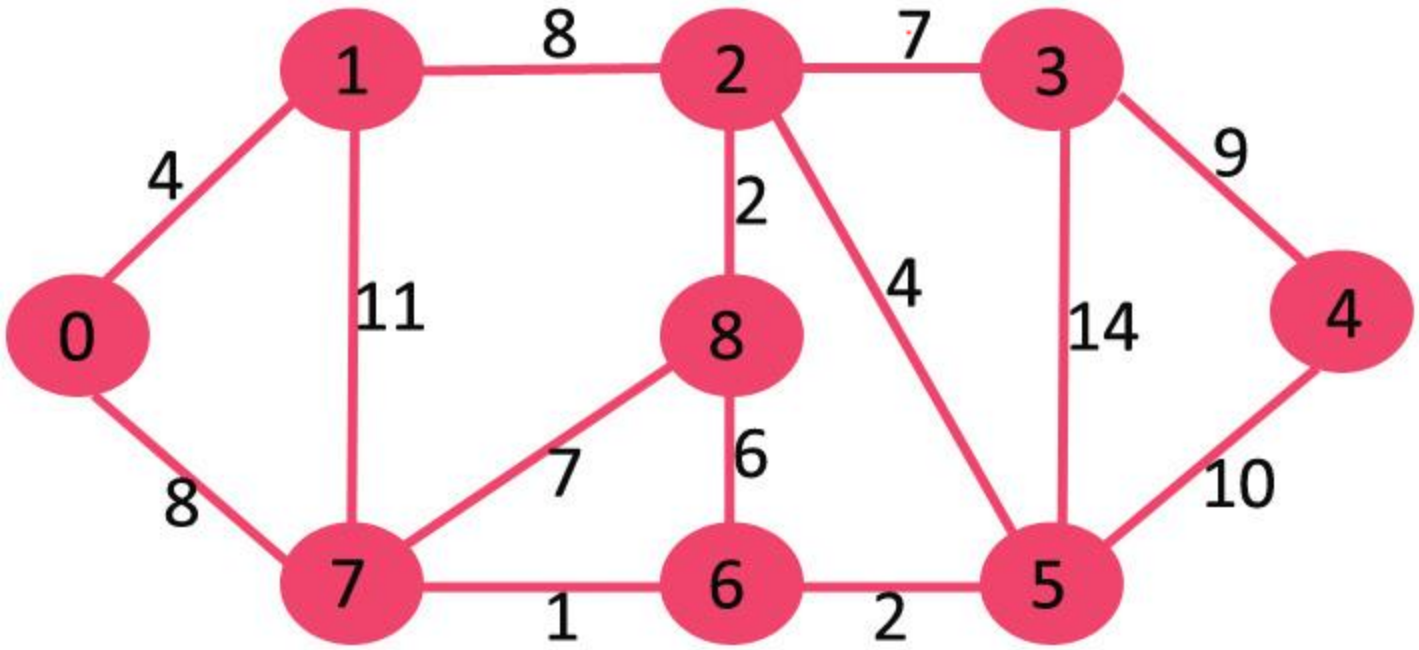


Prim's algorithm

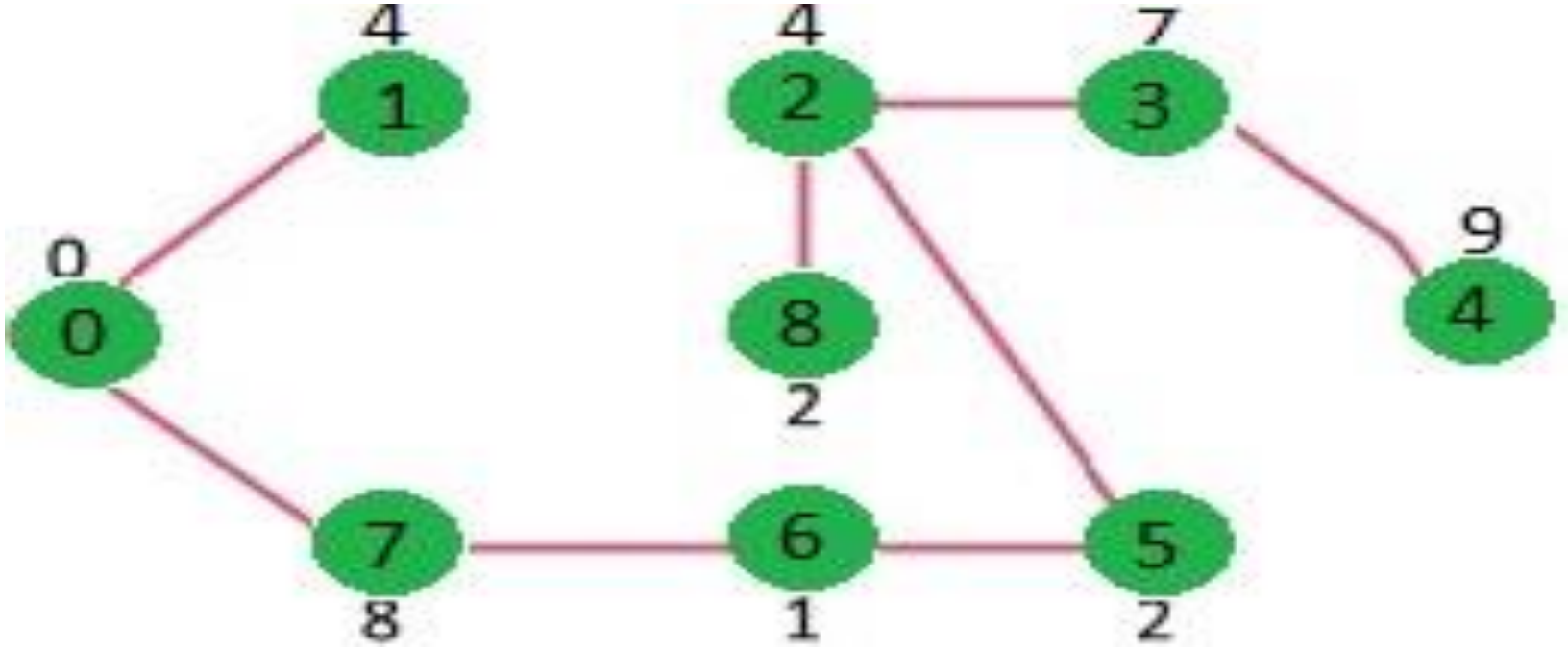
1. Create a set MSTSet that keeps track of vertices already included in MST.
2. Assign a key value to all vertices in the input graph. Initialize all key values as INFINITE. Assign key value as 0 for the first vertex so that it is picked first.
3. While MSTSet doesn't include all vertices :
   a) Pick a vertex u which is not there in MSTSet and has minimum key value
   b) Include u to MSTSet .
   c) Update key value of all adjacent vertices of u.

- To update the key values, iterate through all adjacent vertices. For every adjacent vertex v, if weight of edge w(u,v) is less than the previous key value of v , update the key value as w(u,v).

Example



Minimum cost spanning tree



Algorithm PrimMST(Graph[V][V])

{

parent[V]; // Array to store constructed MST

key[V]; // Key values used to pick minimum weight edge

MSTSet[V]; // To represent set of vertices included in MST

// Initialize all keys as INFINITE

```
for (i = 0; i < V; i++)
 {  key[i] = MAX,
 MSTSet[i] = false;
 }
```

// Always include first 1st vertex in MST &Make key 0 so that this vertex is picked as first vertex.

key[0] = 0;

parent[0] = -1; // First node is always root of MST

```
// The MST will have V vertices
 for (int i= 0; i < V ; i++)
{
// Pick the minimum key vertex from the set of vertices not yet included in   MST
 int u = minKey();
// Add the picked vertex to the MST Set
  MSTSet[u] = true;
// Update key value and parent index of  the adjacent vertices of the picked vertex.
// Consider only those vertices which are  not  yet included in MST
            for (int v = 0; v < V; v++)
            {
             // graph[u][v] is non zero only for adjacent vertices of m
            // MSTSet[v] is false for vertices not yet included in MST
             // Update the key only if graph[u][v] is smaller than key[v]
            if (Graph[u][v] && MSTSet[v] == false && Graph[u][v] < key[v])
              parent[v] = u, key[v] = Graph[u][v];
            }
}
}
```

$$T(n) = V + 1 + V(V+1) + V + 1 + C$$

$$= V + 1 + V^2 + 1 + V + 1 + C$$

$$= V^2 + 2V + C$$

$$T(n) = O(V^2)$$

Prim’s algorithm

- Time Complexity $T(n)=O(V^2)$,If the input graph is represented using adjacency matrix.
- If the input graph is represented using adjacency list, then the time complexity T(n)=0(E log V)

Kruskal's Algorithm

Steps for finding MST using Kruskal's algorithm:

1. Sort the edges in ascending order of their cost.
2. Add the cheapest edge that does not create a cycle
3. Repeat step 2 until all vertices are visited in the tree.

```
Algorithm Kruskal(Graph[V][V])
{
Sort the edges and form Edge list E.
for(v=0;v<V;v++) //create a singleton  T for all vertices
make_set(v);
T= Ø   // MSTSet  that keeps track of edges  already included in MST
 while(i<V&& E!=Ø )
 {
 Choose an edge(u,v)  and delete the edge (u,v)  from the edge list.
 //check whether the addition forms a cycle
            if(find(u)!=find(v))then  //if not in the same tree
            {
            T=TU(u,v)
            Union(u,v)   //merge tress u and v
            }
            else discard the edge
i=i+1;
}
}
```

→ E log E

→ V+1

→ V

→ V+1

$T(n) = E\log E + V+1 + V + V+1$

$= E\log E + CV + C$

$T(n) = O(E\log E)$

Kruskal's Algorithm

- If the input graph is represented using adjacency matrix, then the time complexity T(n)=O(E log E)
- If the input graph is represented using adjacency list, then the time complexity T(n)=O(E log V)

An optimal randomized Algorithm

- Boruvka's Algorithm is an optimal randomized algorithm and it is slightly similar to Kruskal's Algorithm.
- It is also known as Sollin's algorithm
- Time complexity is O(E log v).

An optimal randomized Algorithm

- Input is a connected, weighted and un-directed graph.

1)Initialize all vertices as individual components (or sets).

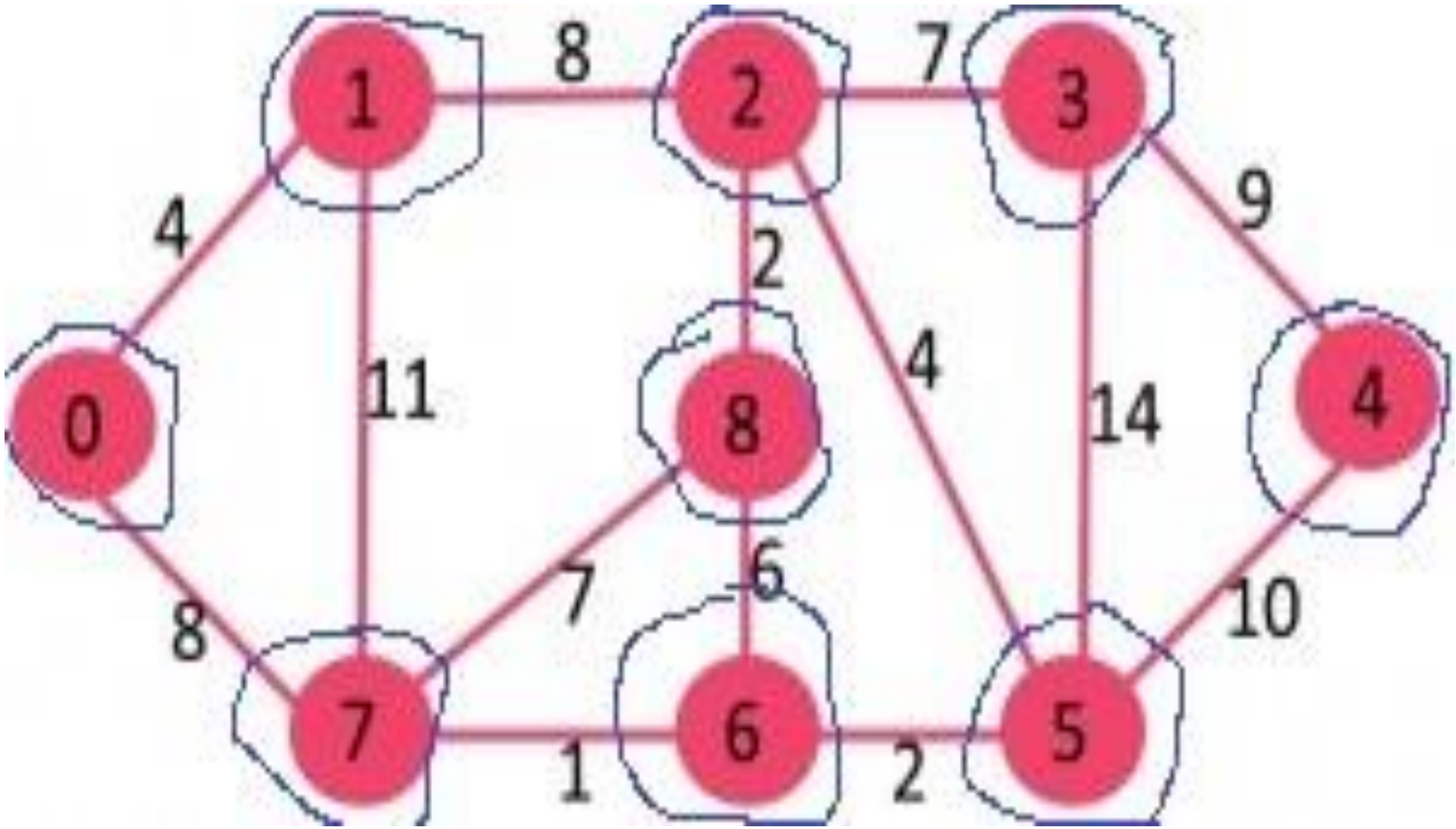
2) Initialize MST as empty.

3) While there are more than one components, do following for each component:

   a) Find the closest weight edge that connects this component to any other component.

   b) Add this closest edge to MST if not already added.

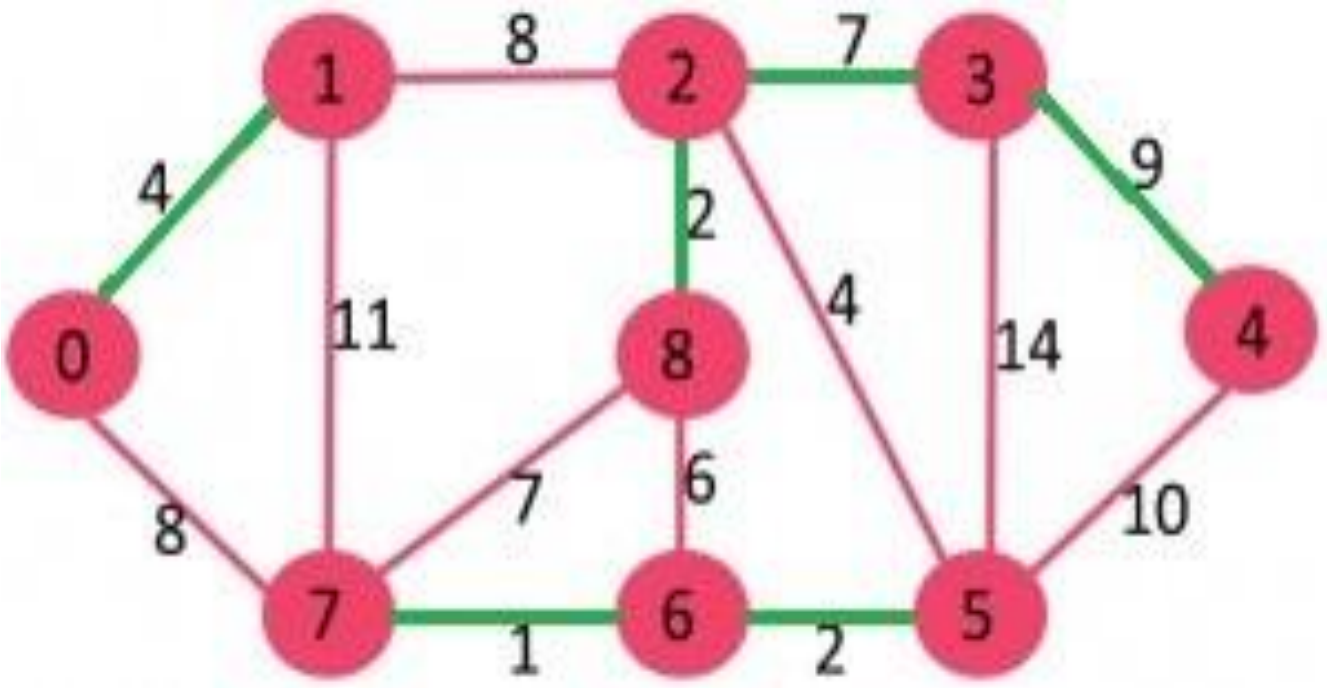
Initially MST is empty. Every vertex is singe component as highlighted in blue color in below diagram.



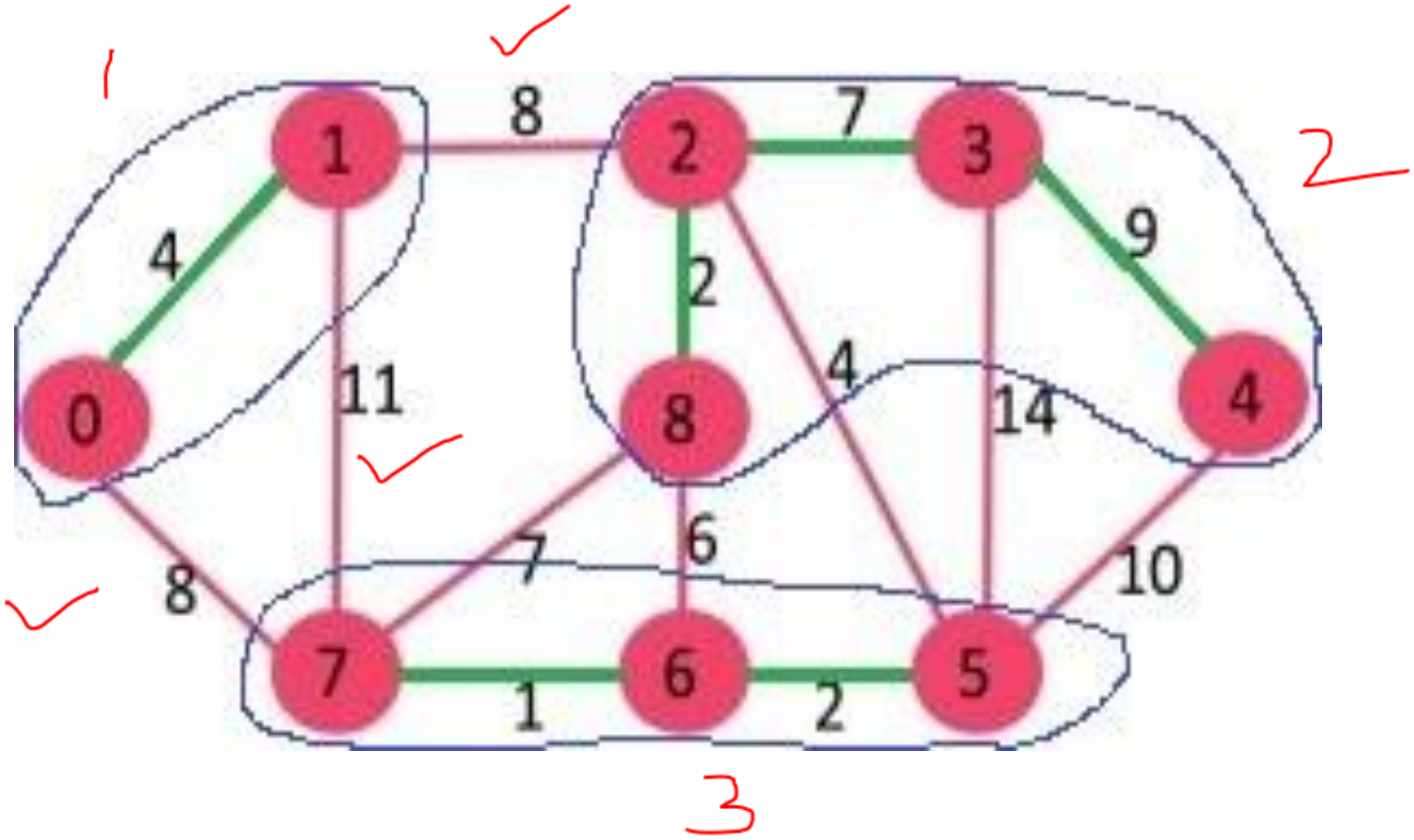
- For every component, find the cheapest edge that connects it to some other component.

Component	Cheapest Edge that connects it to some other component
{0}	0-1
{1}	0-1
{2}	2-8
{3}	2-3
{4}	3-4
{5}	5-6
{6}	6-7
{7}	6-7
{8}	2-8

- Now MST becomes {0-1, 2-8, 2-3, 3-4, 5-6, 6-7}.



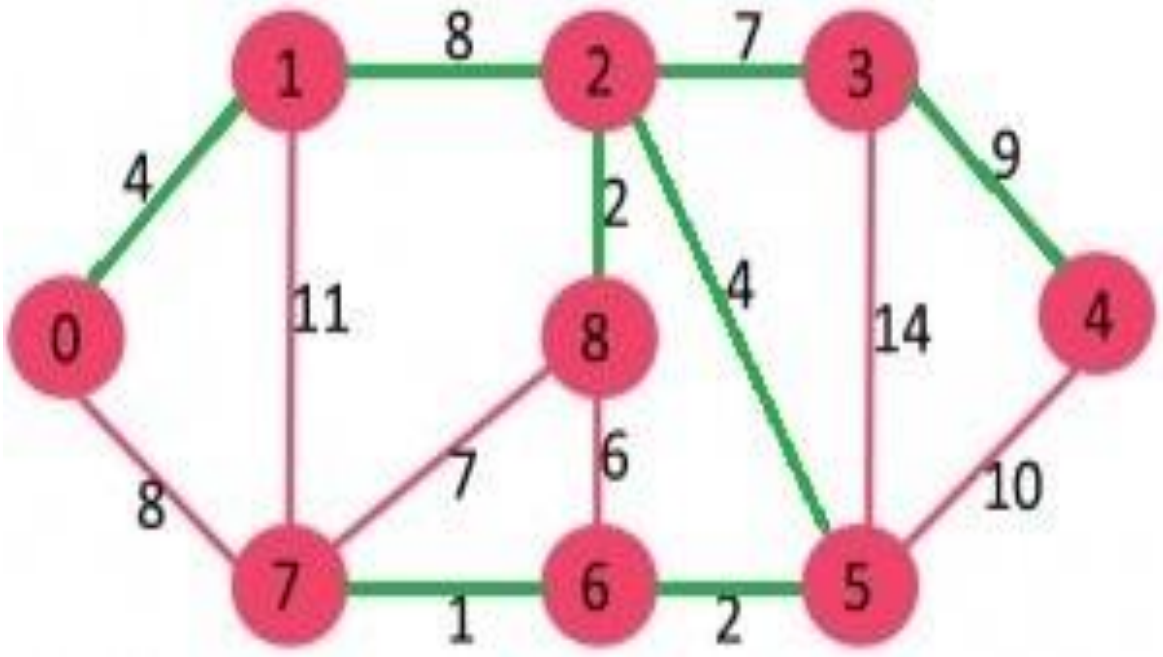
- After above step, components are {{0,1}, {2,3,4,8}, {5,6,7}}. The components are encircled with blue color.



- We again repeat the step, i.e., for every component, find the cheapest edge that connects it to some other component.

Component	**Cheapest Edge that connects it to some other component**
{0,1}	1-2 (or 0-7)
{2,3,4,8}	2-5
{5,6,7}	2-5

- Now MST becomes {0-1, 2-8, 2-3, 3-4, 5-6, 6-7, 1-2, 2-5}



At this stage, there is only one component {0, 1, 2, 3, 4, 5, 6, 7, 8} which has all edges. Since there is only one component left, we stop and return MST.

An optimal randomized Algorithm

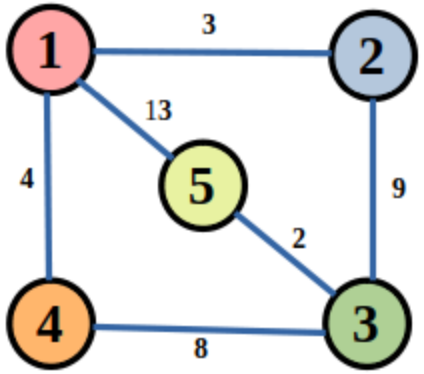


Figure 1

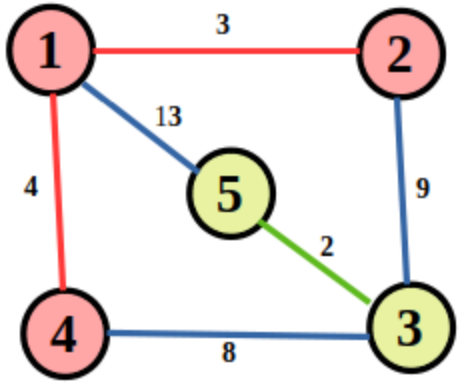


Figure 2

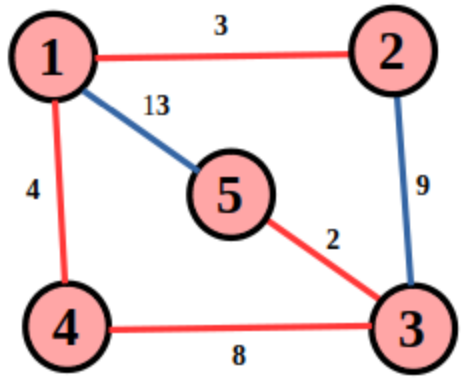
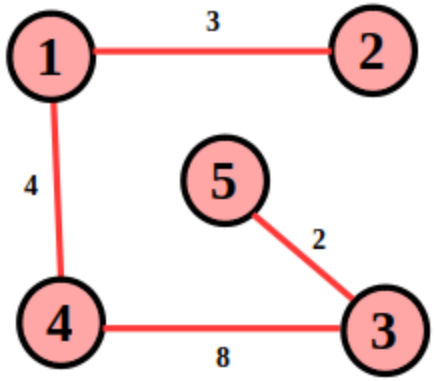


Figure 3



Minimum Spanning Tree

Shortest path problems

- There are four types of shortest path problems

1. Single source shortest path problems
2. Single destination shortest path problems
3. Single pair shortest path problems
4. All pairs shortest path problems.

Single source Shortest path problems

- Given a graph G and a source vertex u in the graph, find shortest paths from a source to all other vertices in the given graph.
- Dijkstra’s Algorithm and Bellman Ford Algorithm are the famous algorithms used for solving single-source shortest path problem.

Single source Shortest path algorithm

- **Greedy algorithm has the following steps**:

1.select a source vertex and assign distance values to all vertices in the input graph. Initialize all distance values as INFINITE. Assign distance value as 0 for the source vertex.

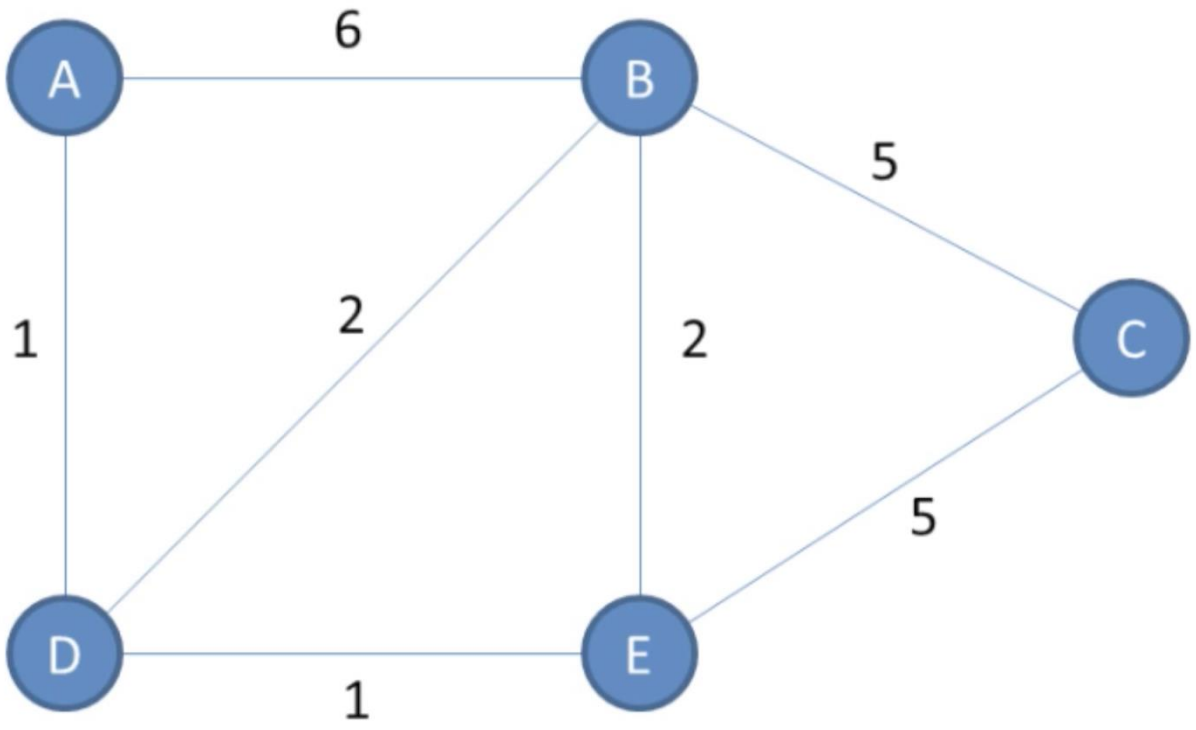
2.visit a vertex u which is not visited and has minimum distance value.

3. Update distance value of all adjacent vertices of u.For every adjacent vertex v

If d(u)+(u,v)<d(v) them d(v)=d(u)+(u,v)

4.repeat step-2&3 until all vertices are visited.

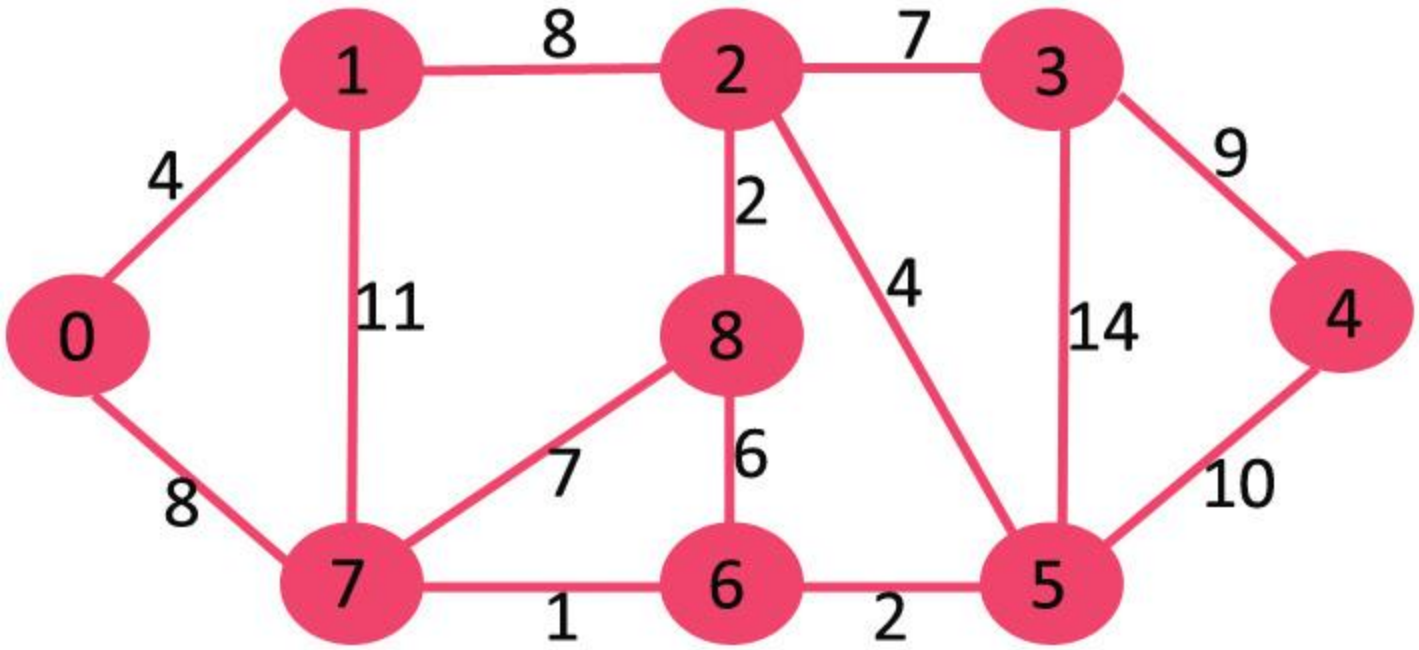
Example

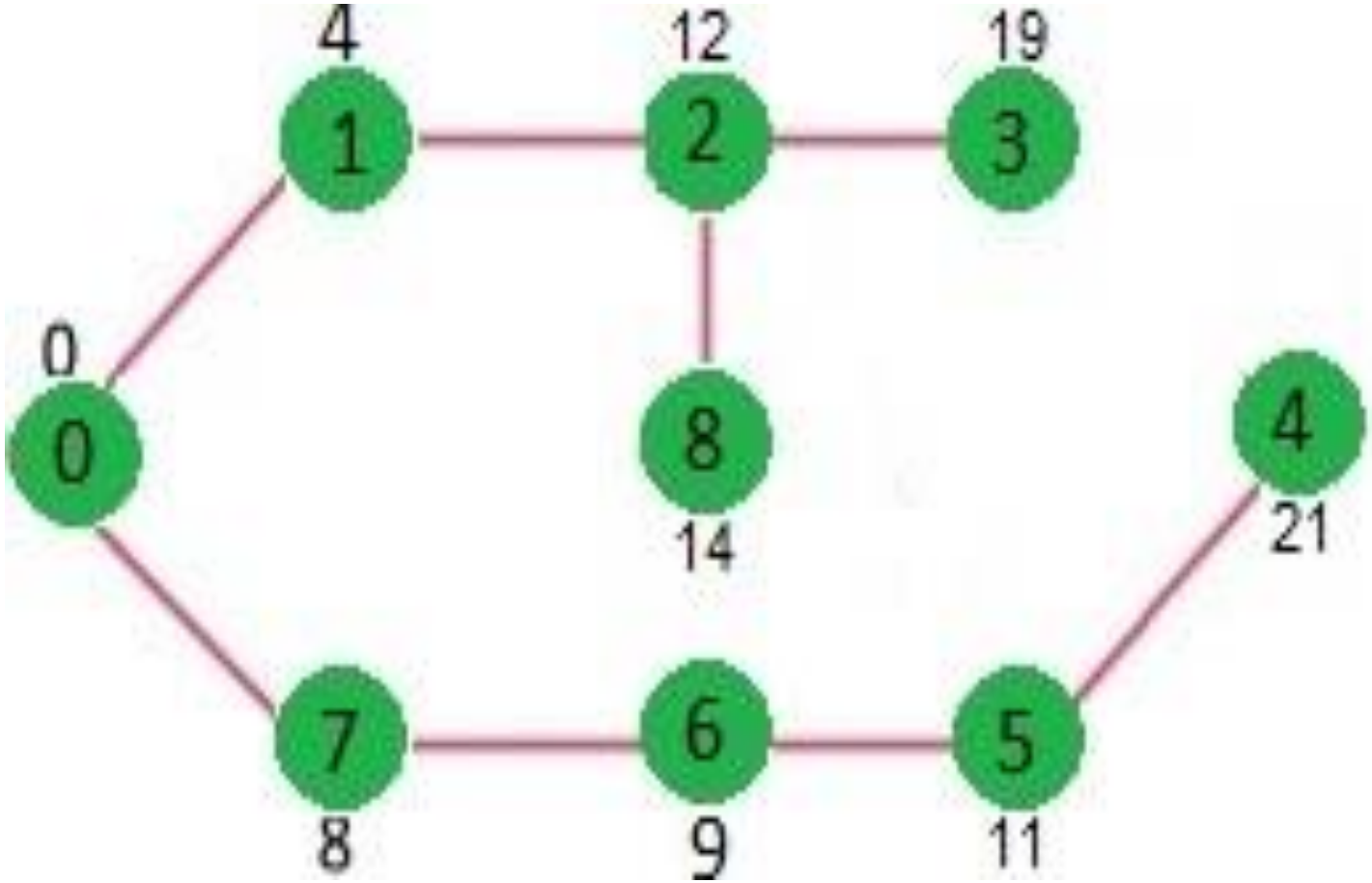


Vertex	Shortest distance from A	Previous vertex
A	0	
B	3	D
C	7	E
D	1	A
E	2	D

Visited = [A, D, E, B, C] Unvisited = []

Example




4
12
19
1
2
3
0
0
8
4
14
21
7
6
5
8
9
11

```
Algorithm Dijk(Graph[V][V],src )
{
dist[V]; // Array to store minimum distance values from source.
SPTSet[V]; // To represent set of vertices included in SPT
// Initialize all keys as INFINITE and SPTSet as false
        for (i = 0; i < V; i++)
         {  dist[i] = MAX,
         SPTSet[i] = false;
        }
 / /Distance of source vertex from itself is always 0
  dist[src] = 0;
```

```
//Find shortest path for all vertices
for (int i= 0; i < V ; i++)
      {
      // Pick the minimum distance vertex from the set of vertices not  yet visited.
    //u is always equal to src in the first iteration.
       int u = minDistance();
// Mark the picked vertex as visited.
       SPTSet[u] = true
     // Update dist value of the adjacent vertices of the picked vertex.
       for (int v = 0; v < V; v++)
            {
           // Update dist[v] only if is not in SPTSet, there is an edge from u to v, and total
            //weight of path from src to  v through u is  smaller than current value of dist[v]
      if (Graph[u][v] &&SPTSet[v]==false &&  dist[u] + graph[u][v] <  dist[v])
              dist[v] = dist[u] + graph[u][v];
           }
  }
 }
```

Time complexity

- Time Complexity $T(n)=O(V^2)$,If the input graph is represented using adjacency matrix.
- If the input graph is represented using adjacency list, then the time complexity T(n)=0(E log V)

Optimal Merge Patterns

- In this problem we should merge a set of sorted files of different length into a single sorted file. We need to find an optimal solution, where the resultant file will be generated in minimum time(minimum no. of comparisons).
- If the number of sorted files are given, there are many ways to merge them into a single sorted file. Different pairings require differing amounts of computing time.
- If we have two sorted files containing n and m records respectively then they could be merged together, to obtain one sorted file in time **O(n+m)**.
- When more than two sorted flies are to be merged together the merge can be accomplished by repeatedly merging sorted files in pairs(2-way merge pattern).
- The two way merge patterns can be represented by **binary merge tree**.

Optimal Merge Patterns

- **The greedy method for optimal merge pattern has the following steps**:

  1.Let x1 and x2 be the two smallest files.

  2.create a new file x with x1 and x2 as the left and right children and update the size of the x.

  3.repeat step-1&2 till only one file is left.

  4.retutn file x.

Optimal Merge Patterns

- Find the optimal merge pattern for the following files:
- 20, 30, 10, 5 and 30

The merging cost = 15+35+60+95
=205

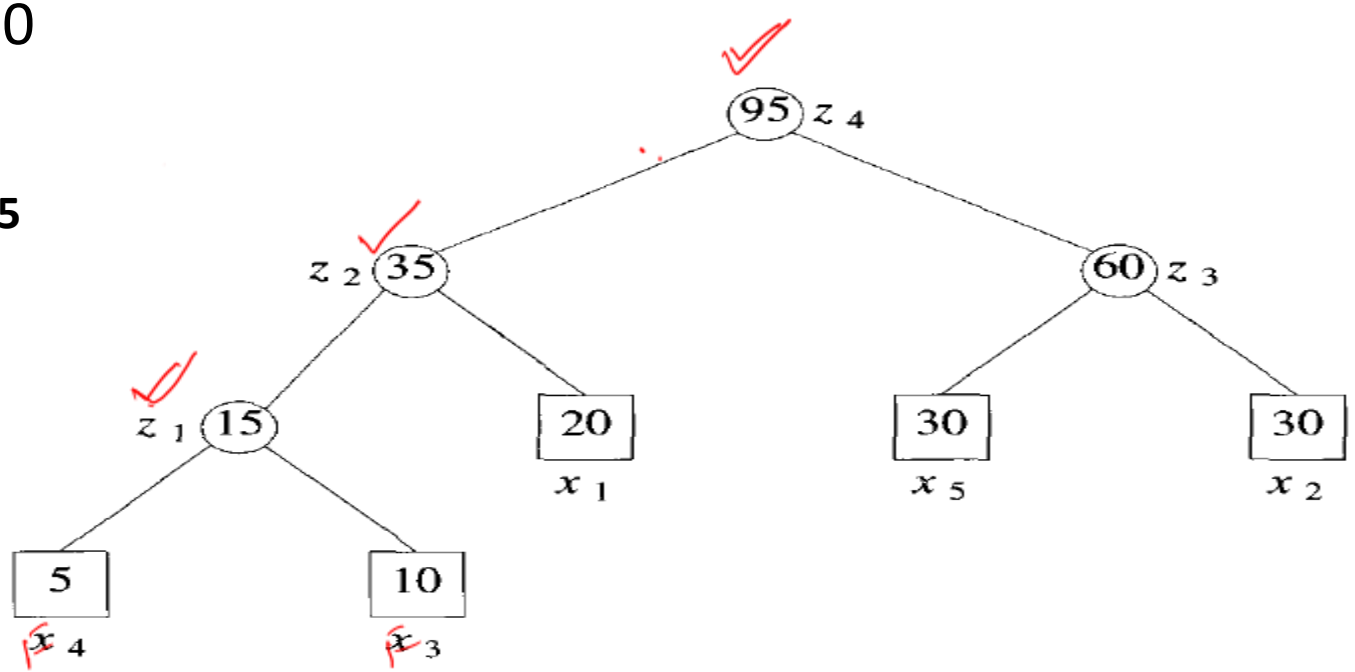


Figure 4.11 Binary merge tree representing a merge pattern

- Find optimal merge pattern for the following files:

5, 3, 2, 7, 9, 13

5, 5, 7, 9, 13

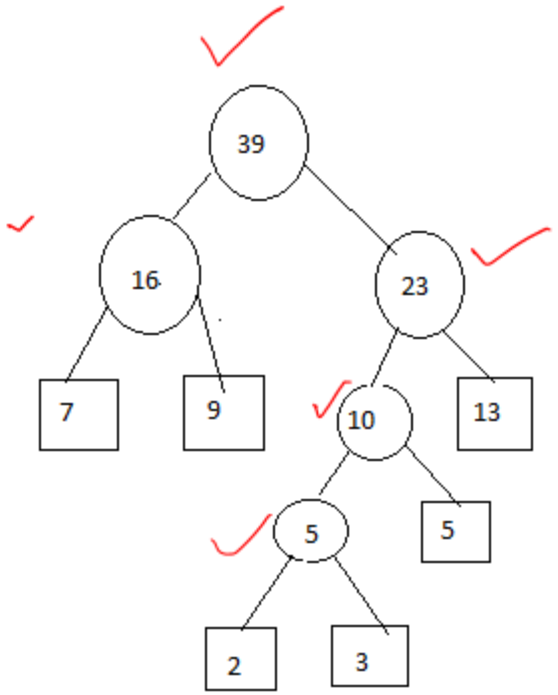
10, 7, 9, 13

10, 16, 13

16, 23

39

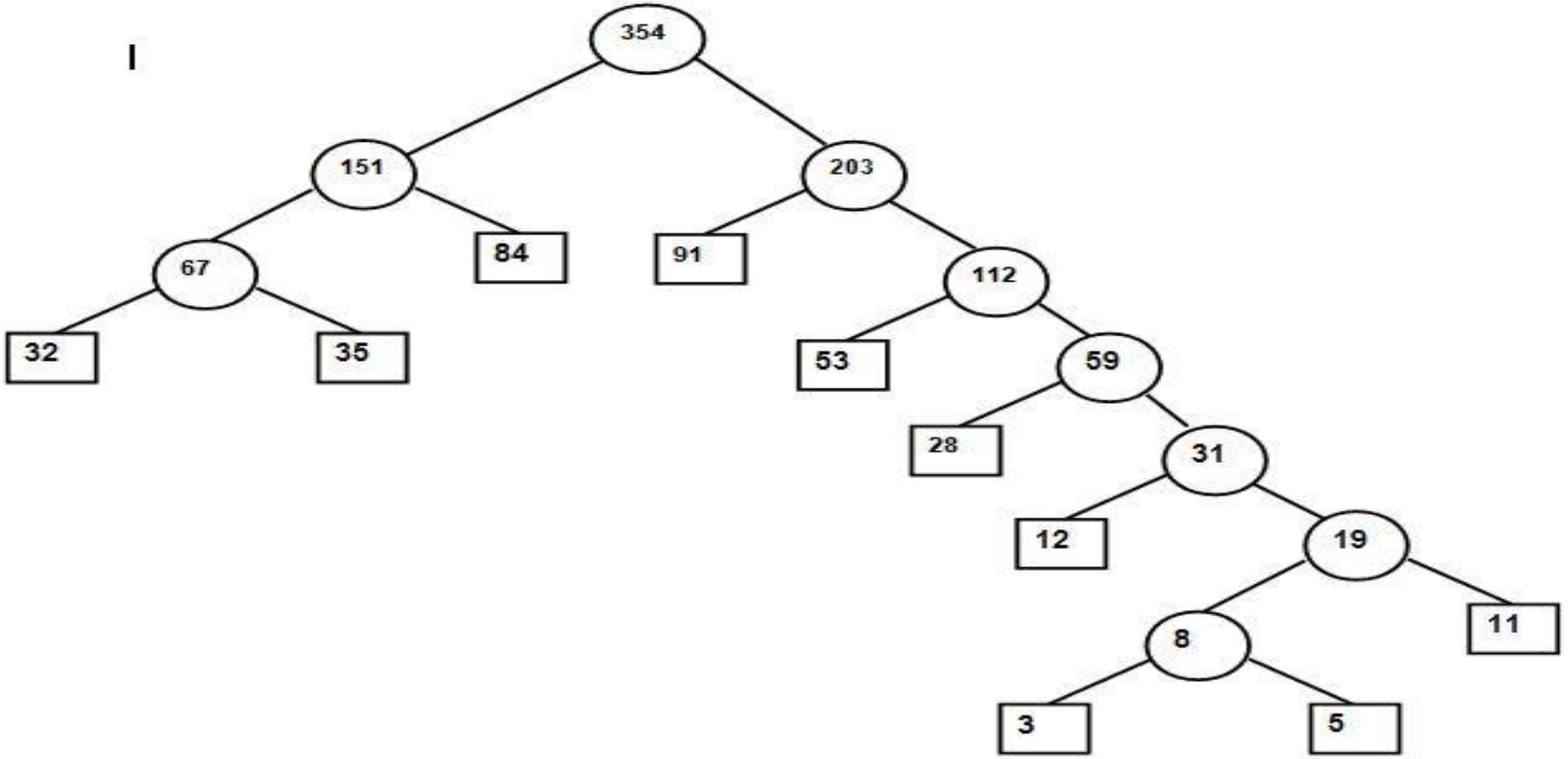
The merging cost = 5 + 10 + 16 + 23 + 39 = 93



- **Find optimal merge pattern for the following files:**

3 5 7 9 12 14 15 17

Find the optimal binary merge tree (pattern) for ten files whose lengths are:
28, 32, 12,5,84,53,91, 35, 3 and 11



```
treenode = record {
    treenode * lchild; treenode * rchild;
    integer weight;
};

Algorithm Tree(n)
// list is a global list of n single node
// binary trees as described above.
{
    for i := 1 to n - 1 do
    {
        pt := new treenode; // Get a new tree node.
        (pt → lchild) := Least(list); // Merge two trees with
        (pt → rchild) := Least(list); // smallest lengths.
        (pt → weight) := ((pt → lchild) → weight)
                        +((pt → rchild) → weight);
        Insert(list, pt);
    }
    return Least(list); // Tree left in list is the merge tree.
}
```

$T(n) = n + 2(n^2 - n)$

$T(n) = 2n^2 - n + C$

$T(n) = O(n^2)$

→ n

→ $n(n-1)$

$n(n-1)$

Time complexity

- Time Complexity $T(n)=O(n^2)$.
- If the list is represented as minheap then the time complexity
- $T(n)=0(n \log n)$